

Лекция 7. Паросочетания. Алгоритм Куна.
Вершинное покрытие и независимое
множество. Введение в потоки. Алгоритм
Форда — Фалкерсона

1 Паросочетания. Основные понятия. Теорема Бержа

Введение. Продолжая изучать графы, рассмотрим последнюю разновидность графов — двудольные графы. В них нас в первую очередь будут интересовать паросочетания, независимые множества и вершинные покрытия.

Определение 1.1. Паросочетанием (*matching*) в неориентированном графе $G = (V, E)$ назовём такое множество рёбер $M \subseteq E$, что не найдётся двух рёбер из M с общей вершиной.

Определение 1.2. Размером паросочетания будем называть $|M|$.

Определение 1.3. Паросочетание M называется *максимальным*, если не существует паросочетания M' такого, что $|M| < |M'|$.

Определение 1.4. Паросочетание M называется *неувеличиваемым*, если не существует ребра $e \in E \setminus M$ такого, что $M \cup \{e\}$ тоже паросочетание.

Определение 1.5. Будем обозначать максимальное паросочетание в графе G как MM_G (max matching) или просто MM .

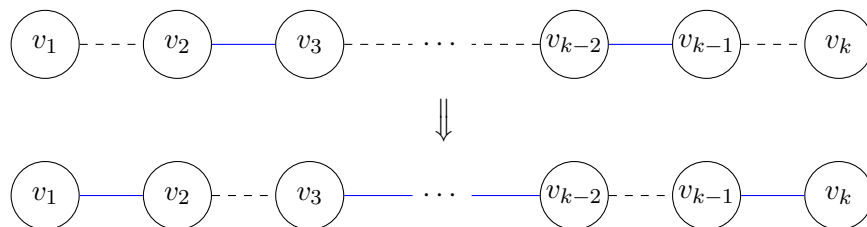
Далее нас будет интересовать задача поиска максимального паросочетания в двудольном графе, потому что минимальное паросочетание — это паросочетание размера 0.

Замечание 1.6. В рамках курса мы будем рассматривать алгоритмы поиска максимального паросочетания только в двудольных графах. Для произвольных графов используется алгоритм сжатия соцветий.

Определение 1.7. Вершина называется *насыщенной паросочетанием M* , если она является концом какого-то ребра из M .

Определение 1.8. Увеличивающая цепь относительно паросочетания M — это путь $p = (v_1, \dots, v_k)$ такой, что $(v_1, v_2) \notin M, (v_2, v_3) \in M, (v_3, v_4) \notin M, \dots, (v_{k-1}, v_k) \notin M$, при этом вершины v_1 и v_k не насыщены M .

Определение 1.9. Операцией чередования вдоль увеличивающей цепи $p = (v_1, \dots, v_k)$ назовём замену следующего вида:



То есть теперь $(v_1, v_2) \in M, (v_2, v_3) \notin M, (v_3, v_4) \in M, \dots, (v_{k-1}, v_k) \in M$.

Замечание 1.10. При операции чередования вдоль увеличивающей цепи размер паросочетания увеличивается на 1.

Теорема 1.11 (Бержа). Паросочетание M в двудольном графе $G = (V, E)$ максимально тогда и только тогда, когда относительно M нет увеличивающих цепей.

Доказательство. $\Rightarrow$ Пусть паросочетание M в двудольном графе максимально. От противного предположим, что относительно M есть увеличивающая цепь p . Проведём операцию чередования вдоль цепи p и получим паросочетание большего размера. Следовательно, исходное паросочетание было не максимальным — противоречие. Значит, относительно M нет увеличивающих цепей.

$\Leftarrow$ Пусть относительно паросочетания M нет увеличивающих цепей. Докажем, что M — максимальное паросочетание. От противного пусть M — не максимальное паросочетание. Тогда существует паросочетание M' такое, что $|M'| > |M|$. Пусть $Q = M \Delta M'$ — симметрическая разность паросочетаний. Докажем две леммы.

Лемма 1.12. В графе $G = (V, Q)$ степень каждой вершины не превосходит 2.

Доказательство. Действительно, поскольку $Q = M \Delta M'$, каждая вершина могла быть насыщена не более, чем каждым из паросочетаний. Поэтому степень каждой вершины не превосходит 2. $\square$

Лемма 1.13. Если в графе степень каждой вершины не превосходит 2, то его рёбра разбиваются на непересекающиеся по вершинам и рёбрам пути и циклы.

Доказательство. Будем действовать по следующему алгоритму:

1. Удалим из графа все изолированные вершины. Это не изменит множество рёбер.
2. Удаляем все пути. Для этого рассматриваем произвольную вершину v_1 , для которой $\deg(v_1) = 1$. Для v_1 существует ребро $e = (v_1, v_2)$. Так как $\deg(v_2) \leq 2$, после удаления e будет выполнено $\deg(v_2) \leq 1$. Если $\deg(v_2) = 0$, то мы выделили путь. Иначе продолжаем строить изолированный путь $p = (v_1, \dots, v_k)$. В конце концов мы найдём все пути и удалим их из графа.
3. После этого остались только вершины степени 2. Рассмотрим произвольную вершину v_0 . Удалим её, для одной из смежных вершин повторим процедуру из пункта 2 для получения изолированного пути $p = (v_1, \dots, v_k)$. Тогда $C = (v_0, v_1, \dots, v_k, v_0)$ — изолированный цикл.

$\square$

Из этих лемм получаем, что Q — это объединение непересекающихся путей и циклов. Рассмотрим отдельно циклы и пути.

1. **Циклы.** В них не могут идти два ребра подряд из одного паросочетания (по определению паросочетания), значит в них поровну рёбер из M и M' и они чередуются.
2. **Пути.** Пусть есть путь нечётной длины $p = (v_1, \dots, v_{2k})$. Рассмотрим два случая:
 - $(v_1, v_2) \in M, (v_2, v_3) \in M', (v_3, v_4) \in M, \dots, (v_{2k-1}, v_{2k}) \in M$. Тогда относительно M' есть увеличивающая цепь p . Следовательно, M' — не максимальное паросочетание (по доказанной необходимости теоремы Бержа).
 - $(v_1, v_2) \in M', (v_2, v_3) \in M, (v_3, v_4) \in M', \dots, (v_{2k-1}, v_{2k}) \in M'$. Тогда относительно M есть увеличивающая цепь p , но M определено как паросочетание без увеличивающих цепей. Получаем, что все пути чётной длины и в них поровну рёбер из M и M' .

В итоге имеем, что во всех циклах поровну рёбер из M и M' и в путях поровну рёбер из M и M' . Кроме того, Q — это объединение непересекающихся по вершинам и рёбрам путей и циклов. Поэтому $|M| = |M'|$. Противоречие с тем, что $|M'| > |M|$ по предположению. Следовательно, M — максимальное паросочетание — что и требовалось доказать. $\square$

2 Алгоритм Куна

Алгоритм 2.1 (Куна). Алгоритм Куна находит максимальное паросочетание в двудольном графе на долях $L, R, L \cup R = V, L \cap R = \emptyset$. Схема алгоритма:

1. Пронумеруем вершины L ;

2. Попробуем для текущей вершины левой доли найти увеличивающую цепь. Если такая цепь найдена, то выполняем чередование вдоль неё.

Увеличивающую цепь можно искать в обычном DFS, который стартует из ненасыщенной вершины $u \in L$. Рассмотрим ребро (u, v) :

1. Если вершина v не насыщена, то найдена цепь (u, v) ;
2. Иначе есть $(v, p) \notin M$. Попробуем найти увеличивающую цепь из p . Если нашли, то $(u, v), (v, p)$ и полученная цепь увеличивающая.

Псевдокод. Приведём код алгоритма Куна. Будем считать, что дан двудольный граф, но его доли заранее не заданы. Для этого раскрасим доли графа в два цвета и будем запускать алгоритм Куна только из вершин одной доли.

```

1 vector<vector<int>> g; // adjacency list
2 vector<int> matching;
3 vector<bool> used;
4 vector<int> color;
5
6 void dfs_color(int u, int c) {
7     color[u] = c;
8     for (int v : g[u])
9         if (color[v] == -1)
10             dfs_color(v, 1 - c);
11 }
12
13 bool dfs(int u) {
14     if (used[u]) return false;
15     used[u] = true;
16     for (int v : g[u]) {
17         if (matching[v] == -1 || dfs(matching[v])) {
18             matching[v] = u;
19             return true;
20         }
21     }
22     return false;
23 }
24
25 int main() {
26     int vertices_count; cin >> vertices_count;
27
28     // read graph
29
30     // color bipartite graph
31     color.resize(vertices_count, -1);
32     for (int i = 0; i < vertices_count; ++i) {
33         if (color[i] == -1) {
34             dfs_color(i, 0);
35         }
36     }
37
38     // Kuhn's algorithm
39     matching.resize(vertices_count, -1);
40     for (int i = 0; i < vertices_count; ++i) {
41         if (color[i] == 0) {
42             used.assign(vertices_count, false);

```

```

43     dfs(i);
44 }
45 }
46
47 // print matching
48 for (int v = 0; v < vertices_count; ++v) {
49     if (matching[v] != -1) {
50         cout << matching[v] << ' ' << v << '\n';
51     }
52 }
53 }

```

Теорема 2.2 (корректность). Алгоритм Куна строит максимальное паросочетание в данном двудольном графе.

Доказательство. Докажем сначала корректность алгоритма поиска одной увеличивающей цепи. Для этого нужно доказать, что DFS находит какую-то увеличивающую цепь, начинающуюся в u , если такая имеется. Доказательство разбивается на две части:

1. DFS находит только увеличивающие пути либо не находит никаких путей.
2. Если существуют увеличивающие пути, начинающиеся в u , тогда DFS, запущенный из u , обязательно найдёт какой-нибудь из этих путей.

Докажем сначала, что DFS находит только увеличивающие пути.

Пусть DFS стартует из ненасыщенной вершины $u \in L$. Алгоритм рассматривает ребро (u, v) . Возможны два случая:

1. Вершина v не насыщена. Тогда путь (u, v) имеет длину 1, начинается и заканчивается в ненасыщенных вершинах и содержит одно ребро, не принадлежащее M . Следовательно, это увеличивающая цепь.
2. Вершина v насыщена. Тогда существует единственная вершина $p \in L$, такая что $(p, v) \in M$. Если из p рекурсивно найдена увеличивающая цепь, то путь $u \rightarrow v \rightarrow p \rightarrow \dots$ является увеличивающей: $(u, v) \notin M, (v, p) \in M$ и так далее. Конец найденной рекурсивно цепи ненасыщен по условию выхода из DFS.

Следовательно, алгоритм DFS либо завершается, не найдя никакого пути, либо находит увеличивающую цепь.

Покажем теперь, что если существует увеличивающая цепь, начинающаяся в u , то DFS её найдёт.

Рассмотрим любой такой путь:

$$u = u_0, v_0, u_1, v_1, \dots, u_k, v_k$$

Где $u_i \in L, v_i \in R$, рёбра $(u_i, v_i) \notin M$, рёбра $(u_{i+1}, v_i) \in M$ для $i \in \{0, \dots, k-1\}$, вершина v_k ненасыщена. DFS из вершины u_0 перебирает все рёбра (u_0, v) . В частности, он когда-нибудь рассмотрит ребро (u_0, v_0) .

- Если v_0 ненасыщена, цепь найдена сразу;
- Иначе v_0 насыщена ребром $(u_1, v_0) \in M$, и алгоритм рекурсивно попытается найти цепь из u_1 .

Далее тот же аргумент повторяется для $u_1, u_2, \dots$. Так как цепь конечна, в конце алгоритм дойдёт до v_k , которая ненасыщена, и найдёт увеличивающую цепь. Чтобы рекурсия не заиклилась, в DFS помечаются посещённые вершины левой доли. Это запрещает повторные заходы в уже посещённые вершины.

Итак, DFS корректно решает задачу:

- Если он нашёл цепь — она увеличивающая;
- Если он не нашёл цепь — увеличивающей цепи из данной вершины u не существует.

Чтобы довести доказательство до конца, осталось заметить ещё одно свойство построенного паросочетания после обработки первых i вершин левой доли: для каждой из уже обработанных вершин $u \in L$ не существует увеличивающей цепи, начинающейся в u , относительно текущего паросочетания M .

Действительно, когда вершина u обрабатывается, алгоритм либо находит увеличивающую цепь из u и увеличивает паросочетание, либо утверждается, что такой цепи нет. После увеличения паросочетания вершина u становится насыщенной, поэтому из неё уже не может начинаться увеличивающая цепь. Для ранее обработанных вершин свойство также сохраняется: если вершина насыщена, то она не может быть началом увеличивающего пути; если же из неё не было цепи на момент обработки и потом она стала насыщенной, тем более. В частности, после завершения работы алгоритма для каждой вершины $u \in L$ не существует увеличивающей цепи, начинающейся в u .

Завершаем доказательство. В двудольном графе любая увеличивающая цепь имеет концы в разных долях, так как её длина нечётна и рёбра идут между L и R . Следовательно, один из концов лежит в L , другой — в R . Значит, если после завершения алгоритма не существует увеличивающей цепи, начинающейся ни в одной вершине L , то и в целом не существует никаких увеличивающих путей. Таким образом, после окончания алгоритма относительно полученного паросочетания M нет увеличивающих цепей. По теореме Берга из этого следует, что M — максимальное паросочетание. $\square$

Теорема 2.3 (время работы алгоритма Куна). Алгоритм Куна работает за $O(|V|^3)$. Если отдельно рассмотреть доли L и R , то время работы сводится к $O(|L|^2|R|)$.

Доказательство. Действительно, в схеме алгоритма алгоритм DFS вызывается в цикле по всем вершинам из левой доли. Внешний цикл — $O(|V|)$ итераций, внутренний DFS — $O(|V| + |E|)$ по времени. В результате получаем $O(|V|) \cdot O(|V| + |E|) = O(|V|^2 + |V||E|)$, что на деле часто рассматривают как $O(|V|^3)$ для плотных графов. Мы получили грубую оценку, но можно оценить асимптотику точнее. В алгоритме Куна обход DFS вызывается для каждой вершины одной из долей. Если размеры долей равны $|L|$ и $|R|$, выгоднее запускать DFS от вершин меньшей доли, чтобы было меньше запусков. Пусть $|L| < |R|$, тогда для каждой из $|L|$ вершин первой доли вызывается DFS за $O(|V| + |E|)$. В итоге получаем $O(|L|) \cdot O(|V| + |E|) = O(|L||E|) = O(|L|^2|R|)$, что может быть существенно быстрее, если одна доля сильно меньше другой. $\square$

3 Вершинное покрытие и независимое множество

Определение 3.1. Множество $I \subseteq V$ неориентированного графа $G = (V, E)$ называется *независимым множеством* (*independent set*), если $\forall v \in V \forall u \in I \rightarrow (u, v) \notin E$.

Определение 3.2. Множество $C \subseteq V$ неориентированного графа $G = (V, E)$ называется *вершинным покрытием* (*vertex cover*), если $\forall e = (u, v) \in E$ верно, что $u \in C$ или $v \in C$.

Замечание 3.3. Далее будем искать максимальное независимое множество и минимальное вершинное покрытие.

Замечание 3.4. В произвольном графе задачи поиска минимального вершинного покрытия или максимального независимого множества являются NP-полными. Мы будем искать их для двудольных графов.

Замечание 3.5. Далее договоримся обозначать максимальное независимое множество графа G как IS_G или просто IS . Аналогично, минимальное вершинное покрытие будем обозначать как VC_G или VC .

Теорема 3.6 (связь вершинного покрытия и независимого множества). Пусть $C \subseteq V$ — вершинное покрытие. Тогда $S = V \setminus C$ — независимое множество. Верно и обратное утверждение.

Доказательство. $\Rightarrow$ Пусть $C \subseteq V$ — вершинное покрытие. Это значит, что для любого ребра графа хотя бы один его конец лежит в C . Рассмотрим дополнение $S = V \setminus C$. Докажем, что S — независимое множество, то есть между вершинами множества S нет рёбер. Предположим противное: есть ребро (u, v) , такое что $u \in S, v \in S$. Но тогда ни u , ни v не лежат в C , значит ребро (u, v) не покрыто вершинами из C . Это противоречит тому, что C — вершинное покрытие. Следовательно, S — независимое множество.

$\Leftarrow$ Пусть $S \subseteq V$ — независимое множество. Рассмотрим $C = V \setminus S$. Если бы C не являлось вершинным покрытием, существовало бы ребро (u, v) , у которого оба конца не лежат в C . Тогда оба конца лежат в S , а это невозможно, так как S является независимым множеством. Значит, $V \setminus S$ — вершинное покрытие. $\square$

Теорема 3.7 (связь $|IS_G|$ и $|VC_G|$). В произвольном графе $G = (V, E)$ выполнено $|IS_G| = |V| - |VC_G|$.

Доказательство. Пусть C — минимальное вершинное покрытие. Тогда по предыдущей теореме $V \setminus C$ — независимое множество размера $|V| - |C|$. Предположим, существует независимое множество S большего размера: $|S| > |V| - |C|$. Но тогда по предыдущей теореме $V \setminus S$ — вершинное покрытие размера $|V \setminus S| = |V| - |S| < |C|$, что противоречит минимальности C . Значит, независимого множества больше, чем $V \setminus C$, быть не может и $|IS_G| = |V| - |VC_G|$. $\square$

Постановка задачи. Дан двудольный граф $G = (L \sqcup R, E)$ на долях L и R . Требуется найти VC_G и IS_G .

Алгоритм 3.8. Построим MM_G . Ориентируем рёбра в G так, что из L в R будут вести рёбра НЕ из MM и наоборот. Вызовем DFS от каждой ненасыщенной вершины левой доли. Посещённые вершины левой доли обозначим как L^+ , а непосещённые как L^- , аналогично введём R^+ и R^- . Тогда $VC_G = L^- \sqcup R^+$, а также $IS_G = V \setminus VC_G$.

Обозначим граф, полученный после ориентации рёбер, как $G_O = (L \sqcup R, E_O)$.

Лемма 3.9. В графе G_O нет рёбер из R^+ в L^- и из L^+ в R^- .

Доказательство. Действительно, если бы существовало ребро $(u, v) \in E_O, u \in R^+, v \in L^-$, то DFS, попав в вершину u , прошёл бы по ребру (u, v) и попал бы в вершину v . Но по предположению $v \in L^-$, то есть DFS её не посетил — противоречие. Аналогично и для рёбер из L^+ в R^- . $\square$

Лемма 3.10. В графе G_O нет рёбер из R^- в L^+ .

Доказательство. От противного, пусть $\exists e = (u, v) \in E_O : u \in R^-, v \in L^+$. Заметим, что ребро идёт из доли в R в долю L , а по построению все такие рёбра принадлежат MM_G . Значит, $e \in MM_G$. Тогда вершина v насыщена по определению. По приведённому алгоритму DFS запускался только от ненасыщенных вершин левой доли. Следовательно, вершина v была посещена в ходе рекурсивного вызова DFS (она была посещена, так как $v \in L^+$). Из какой вершины DFS пришёл в вершину v ? Из вершин левой доли не мог, так как граф двудольный. Значит, он пришёл из вершины правой доли по ребру из паросочетания. Но по определению паросочетания в нём нет двух рёбер с вершиной v . Одно из них — ребро (u, v) , второго быть не может. Значит, DFS мог прийти в v только из u . Но $u \in R^-$, то есть u не была посещена в ходе DFS — противоречие с тем, что есть ребро из R^- в L^+ . Что и требовалось доказать. $\square$

Теорема 3.11 (корректность). Приведённый алгоритм корректно находит VC_G и IS_G .

Доказательство. Из ранее доказанных лемм следует:

- нет рёбер из R^+ в L^- ;

- нет рёбер из L^+ в R^- ;
- нет рёбер из R^- в L^+ .

Последние два пункта означают, что между L^+ и R^- вообще нет рёбер в исходном графе G . Тогда $L^+ \sqcup R^-$ — независимое множество, а его дополнение $L^- \sqcup R^+$ — вершинное покрытие. Покажем, что $L^- \sqcup R^+$ является минимальным вершинным покрытием, то есть $VC_G = L^- \sqcup R^+$. Для этого оценим размер вершинного покрытия с двух сторон.

Покажем, что $|VC_G| \geq |MM_G|$. Действительно, любое вершинное покрытие обязано содержать хотя бы по одной вершине из каждого ребра паросочетания. А так как рёбра паросочетания не пересекаются по вершинам, для разных рёбер нужны разные вершины. Значит, в любом вершинном покрытии не меньше $|MM_G|$ вершин, то есть $|VC_G| \geq |MM_G|$.

Покажем, что $|L^- \sqcup R^+| \leq |MM_G|$. Для этого заметим следующее: - В L^- находятся только насыщенные вершины. Действительно, если бы вершина $u \in L^-$ была насыщенной, то DFS запускался бы из неё по алгоритму. Тогда u была бы посещена, то есть $u \in L^+$ — противоречие. - В R^+ находятся только насыщенные вершины. Предположим противное: существует ненасыщенная вершина $v \in R^+$. Тогда она была достигнута в ходе DFS, запущенного от некоторой ненасыщенной вершины левой доли. Рассмотрим путь в дереве DFS от корня до v . Этот путь начинается в ненасыщенной вершине из L , заканчивается в ненасыщенной вершине $v \in R$, а по построению ориентации рёбра на нём чередуются: из L в R идут рёбра не из MM_G , из R в L — рёбра из MM_G . Значит, это увеличивающая цепь относительно MM_G , что противоречит максимальности MM_G по теореме Бержа. Следовательно, все вершины из R^+ насыщены. - Ни одно ребро паросочетания не соединяет вершину из L^- с вершиной из R^+ . Действительно, по построению все рёбра паросочетания ориентированы из R в L . Но по доказанной выше лемме нет рёбер из R^+ в L^- .

Теперь рассмотрим все рёбра паросочетания MM_G . Так как каждая вершина из $L^- \sqcup R^+$ насыщена (по только что доказанным пунктам), каждой такой вершине соответствует некоторое ребро из MM_G . Причём одно ребро паросочетания не может одновременно соответствовать вершине из L^- и вершине из R^+ , потому что тогда его концы лежали бы соответственно в L^- и R^+ , что невозможно по последнему доказанному пункту. Следовательно, каждому ребру из MM_G соответствует не более одной вершины из множества $L^- \sqcup R^+$. Значит, $|L^- \sqcup R^+| \leq |MM_G|$.

Мы получили двустороннюю оценку: $|L^- \sqcup R^+| \leq |MM_G| \leq |VC_G|$. Но $L^- \sqcup R^+$ — вершинное покрытие, поэтому $|VC_G| \leq |L^- \sqcup R^+|$, значит, $|L^- \sqcup R^+| \geq |MM_G|$. Отсюда $|L^- \sqcup R^+| = |MM_G|$ и $L^- \sqcup R^+$ — минимальное вершинное покрытие, то есть $VC_G = L^- \sqcup R^+$. По теореме о связи вершинного покрытия и независимого множества дополнение к вершинному покрытию является независимым множеством. Тогда $IS_G = V \setminus VC_G = L^+ \sqcup R^-$ — что и требовалось доказать. $\square$

4 Потоки. Базовые понятия. Теорема Форда — Фалкерсона

Введение. Мы дошли до супремума нашего курса — потоков в графах. Это действительно та тема, которую не получится «объехать на кривой козе». Это большая теория, где новые определения строятся на старых, поэтому хочется пожелать сил всем тем, кто читает этот параграф за пару дней до экзамена.

Определение 4.1. Пара рёбер $(u, v) \in E$ и $(v, u) \in E$ в ориентированном графе $G = (V, E)$ называются *антипараллельными*.

Определение 4.2. *Сетью* назовём кортеж $\mathcal{N} = (G, c, s, t)$, где $G = (V, E)$ — ориентированный граф, в котором нет антипараллельных рёбер, $c : E \rightarrow \mathbb{R}_{\geq 0}$ — *вместимость* (пропускная способность ребра) или *capacity* (неотрицательное число для каждого ребра) и $s \neq t \in V$ — *исток* и *сток* соответственно.

Замечание 4.3. Заранее для удобства расширим пропускную способность $c : V \times V \rightarrow \mathbb{R}_{\geq 0}$. Если $(u, v) \notin E$, положим $c(u, v) = 0$.

Определение 4.4. *Потоком* в сети назовём функцию $f : V \times V \rightarrow \mathbb{R}_{\geq 0}$, для которой выполняются следующие свойства:

1. $\forall u, v \in V \hookrightarrow 0 \leq f(u, v) \leq c(u, v)$ — свойство ограниченности потока;
2. $\forall u \in V \setminus \{s, t\} \hookrightarrow \sum_{v \in V} f(u, v) = \sum_{v \in V} f(v, u)$ — свойство сохранения потока.

На рабоче-крестьянском языке: представим граф как трубопровод. Пропускная способность ребра — это сколько литров воды в секунду по нему может течь. Поток — это сколько воды у вас прямо сейчас течёт по каждой трубе. Чтобы трубы не прорвало, по ним не может течь воды больше, чем пропускная способность (первое требование к потоку). Ясно, что вода нигде не должна накапливаться и появляться из ниоткуда, кроме истока и стока. Можно думать, что вода вытекает из истока и втекает в сток. Понятно, сколько вытекло из истока, столько должно втечь в сток.

Замечание 4.5. Если $(u, v) \notin E$, то $0 \leq f(u, v) \leq c(u, v) = 0$. Поэтому доопределение функции c полезно.

Замечание 4.6. Существует несколько способов определять поток в сети. Мы рассматриваем случай, когда поток, протекающий по ребру, неотрицателен. Чаще поток определяют через кососимметрию, но такая модель тоже допустима.

Замечание 4.7. В таком определении потока необязательно требовать отсутствия антипараллельных рёбер, это замечание будет важно при введении определения дополняющего потока.

Определение 4.8. Пусть в сети $\mathcal{N}$ зафиксирован поток f , тогда *величиной потока* называют

$$|f| = \sum_{v \in V} f(s, v) - \sum_{v \in V} f(v, s)$$

На рабоче-крестьянском языке: величина потока — это сколько воды в секунду уходит из истока — разность, сколько вытекло и сколько в него втекло.

Определение 4.9. *Задача о максимальном потоке* — это задача поиска в сети такого потока f , что $|f|$ максимален.

Замечание 4.10. Если в ориентированном графе есть антипараллельные рёбра (u, v) и (v, u) , то вводят фиктивную вершину w и рёбра (u, w) и (w, v) , причём $c(u, w) = c(w, v) = c(u, v)$.

Замечание 4.11. Любому потоку в сети до модификации выше соответствует поток той же величины, проходящий по тем же рёбрам (кроме новых и удалённых).

Замечание 4.12. Иногда возникает необходимость ввести несколько истоков $s_1, \dots, s_k$ и несколько стоков $t_1, \dots, t_k$. Тогда введём фиктивный мультиисток s и рёбра (s, s_i) такие, что $c(s, s_i) = \infty$ и фиктивный мультисток t и рёбра (t_i, t) такие, что $c(t_i, t) = \infty$. Данная модификация сети выполняет сведение задачи с множеством стоков и истоков к классической сети. Где-то мы это уже видели...

Определение 4.13. *Остаточной сетью* для сети $\mathcal{N} = (G = (V, E), c, s, t)$ и потока f в $\mathcal{N}$ назовём объект $\mathcal{N}_f = (G' = (V, E_f), c_f, s, t)$, такой что

$$\forall u \in V, \forall v \in V \quad c_f(u, v) = \begin{cases} c(u, v) - f(u, v), & (u, v) \in E \\ f(v, u), & (v, u) \in E \\ 0, & \text{иначе} \end{cases}$$

Где $E_f = \{(u, v) \mid u, v \in V, c_f(u, v) > 0\}$. В остаточном графе допускаются антипараллельные рёбра. **На рабоче-крестьянском языке:** первое, на что нужно обратить внимание — это на то, что множество рёбер другое. Также здесь функция c_f определена для всех пар вершин. Для прямого ребра, которое было в графе (напоминаем, граф ориентированный!), новая пропускная способность в остаточной сети носит характер «сколько ещё воды можно долить вдоль этого ребра». То есть если у нас ребро $u \rightarrow v$ вмещает $c(u, v) = 5$, а течёт $f(u, v) = 3$, то долить мы ещё можем $c_f(u, v) = 5 - 3 = 2$. Далее, для «обратных» рёбер (v, u) таких, что $(u, v) \in E$, пропускная способность ребра (v, u) носит характер «сколько потока вдоль ребра (u, v) можно отменить». Например, если вдоль ребра $u \rightarrow v$ течёт поток $f(u, v) = 2$, то $c_f(v, u) = f(u, v) = 2$. То есть в обратную сторону мы можем отменить 2 литра воды. Ещё следует обратить внимание, что в остаточной сети присутствуют только рёбра с пропускной способностью, отличной от нуля.

Замечание 4.14. В остаточной сети пропускные способности всех рёбер строго положительные, но это не сеть по определению: в ней могут быть антипараллельные рёбра.

Определение 4.15. Дополняющим потоком для сети $\mathcal{N}$ и потока f определим поток f' в остаточной сети $\mathcal{N}_f$.

Определение 4.16. Сложение потоков $f + f'$ определяется следующим образом:

$$\forall u, v \in V \quad (f + f')(u, v) = \begin{cases} f(u, v) + f'(u, v) - f'(v, u), & (u, v) \in E \\ 0, & \text{иначе} \end{cases}$$

Теорема 4.17. Сумма потока f и дополняющего потока f' является потоком.

Доказательство. Проверим определение потока:

- Неотрицательность:

$$\begin{aligned} (f + f')(u, v) &= f(u, v) + f'(u, v) - f'(v, u) \geq \\ &\geq f(u, v) + f'(u, v) - c_f(v, u) = \\ &= f(u, v) + f'(u, v) - f(u, v) = f'(u, v) \boxed{\geq 0} \end{aligned}$$

- Ограничение пропускной способности:

$$\begin{aligned} (f + f')(u, v) &= f(u, v) + f'(u, v) - f'(v, u) \leq \\ &\leq f(u, v) + f'(u, v) \leq \\ &\leq f(u, v) + c_f(u, v) = \\ &= f(u, v) + c(u, v) - f(u, v) = c(u, v) \end{aligned}$$

- Свойство сохранения потока:

$$\begin{aligned} \forall u \in V \setminus \{s, t\} \hookrightarrow \sum_{v \in V} (f + f')(u, v) &= \sum_{v \in V} (f(u, v) + f'(u, v) - f'(v, u)) = \\ &= \sum_{v \in V} f(u, v) + \sum_{v \in V} f'(u, v) - \sum_{v \in V} f'(v, u) = \\ &= \left[\text{для } f \text{ и } f' \text{ верно сохранение потока} \right] = \\ &= \sum_{v \in V} f(v, u) + \sum_{v \in V} f'(v, u) - \sum_{v \in V} f'(u, v) = \\ &= \sum_{v \in V} (f(v, u) - f'(v, u) - f'(u, v)) = \sum_{v \in V} (f + f')(v, u) \end{aligned}$$

□

Теорема 4.18 (б/д). $|f + f'| = |f| + |f'|$.

Доказательство. Вспомним определение дополняющего потока f' : он добавляет поток по прямым рёбрам (u, v) и отменяет поток по обратным рёбрам (v, u) . Поэтому отмена всего потока f' даст исходный поток f , то есть $|f + f'| - |f'| = |f| \Leftrightarrow |f + f'| = |f| + |f'|$ — что и требовалось доказать. □

Определение 4.19. Разрезом в сети $\mathcal{N} = (G = (V, E), c, s, t)$ назовём пару множеств $\langle S, T \rangle$, такую что:

1. $S \subseteq V \quad \wedge \quad T \subseteq V$;
2. $s \in S, t \in T$;
3. $S \cap T = \emptyset \quad \wedge \quad S \cup T = V$.

Определение 4.20. Поток через разрез $\langle S, T \rangle$ назовём величину

$$f(S, T) = \sum_{u \in S} \sum_{v \in T} f(u, v) - \sum_{u \in S} \sum_{v \in T} f(v, u)$$

На рабоче-крестьянском языке: сколько течёт через разрез — это сколько вытекло из S минус сколько втекло.

Определение 4.21. Вместимостью или пропускной способностью разреза назовём величину

$$c(S, T) = \sum_{u \in S} \sum_{v \in T} c(u, v)$$

На рабоче-крестьянском языке: пропускная способность разреза имеет смысл верхней оценки на то, сколько одновременно может вытекать из S в T .

Определение 4.22. Минимальным разрезом назовём разрез с минимальной пропускной способностью.

Теорема 4.23. Пусть в сети $\mathcal{N}$ задан поток f , тогда для любого разреза $\langle S, T \rangle$ величина потока через него равна $f(S, T) = |f|$.

Доказательство. Аккуратно распишем по определениям. По определению потока для любой вершины $u \notin \{s, t\}$:

$$\sum_{v \in V} f(u, v) - \sum_{v \in V} f(v, u) = 0$$

С другой стороны, по определению величины потока:

$$|f| = \sum_{v \in V} f(s, v) - \sum_{v \in V} f(v, s) =$$

Добавим «умный ноль» из определения потока, перегруппируем и получим требуемое:

$$\begin{aligned}
|f| &= \sum_{v \in V} f(s, v) - \sum_{v \in V} f(v, s) = \\
&= \sum_{v \in V} f(s, v) - \sum_{v \in V} f(v, s) + \sum_{u \in S \setminus \{s\}} \left(\sum_{v \in V} f(u, v) - \sum_{v \in V} f(v, u) \right) = \\
&= \sum_{u \in S} \sum_{v \in V} f(u, v) - \sum_{u \in S} \sum_{v \in V} f(v, u) = [S \sqcup T = V] = \\
&= \sum_{u \in S} \sum_{v \in S} f(u, v) + \sum_{u \in S} \sum_{v \in T} f(u, v) - \sum_{u \in S} \sum_{v \in S} f(v, u) - \sum_{u \in S} \sum_{v \in T} f(v, u) = \\
&= \left[\sum_{u \in S} \sum_{v \in S} f(u, v) = \sum_{u \in S} \sum_{v \in S} f(v, u) \right] = \\
&= \sum_{u \in S} \sum_{v \in T} f(u, v) - \sum_{u \in S} \sum_{v \in T} f(v, u) = f(S, T)
\end{aligned}$$

□

Следствие 4.24. Величина любого потока ограничена пропускной способностью произвольного разреза.

Доказательство. Действительно, по только что доказанной теореме для любого разреза $\langle S, T \rangle$:

$$\begin{aligned}
|f| = f(S, T) &= \sum_{u \in S} \sum_{v \in T} f(u, v) - \sum_{u \in S} \sum_{v \in T} f(v, u) \leq \\
&\leq \sum_{u \in S} \sum_{v \in T} f(u, v) \leq \\
&\leq \sum_{u \in S} \sum_{v \in T} c(u, v) = c(S, T)
\end{aligned}$$

□

Теорема 4.25 (Форда — Фалкерсона). Пусть задана сеть $\mathcal{N} = (G = (V, E), c, s, t)$ и поток f в ней. Следующие утверждения эквивалентны:

1. Поток f в сети максимален;
2. В остаточной сети $\mathcal{N}_f$ нет пути из s в t ;
3. Существует разрез $\langle S, T \rangle$ такой, что $c(S, T) = |f|$.

Доказательство. $\boxed{1 \rightarrow 2}$ От противного, пусть в $\mathcal{N}_f$ есть путь из s в t . Поскольку в $\mathcal{N}_f$ пропускные способности рёбер больше нуля, мы можем пустить по $\mathcal{N}_f$ поток такой, что $|f'| > 0$. По теореме о величине потока суммы, $|f + f'| = |f| + |f'| > |f|$. Противоречие тому, что поток f максимален.

$\boxed{3 \rightarrow 1}$ В предыдущей теореме было доказано, что для любого разреза $\langle S, T \rangle$ величина потока $|f| = f(S, T)$ не превосходит $c(S, T)$. А поскольку существует разрез $\langle S, T \rangle$ такой, что $|f| = c(S, T)$, увеличение величины потока повлечёт увеличение величины потока через разрез, и будет $|f| > c(S, T)$. Противоречие, следовательно, поток f нельзя увеличить и он максимален.

$\boxed{2 \rightarrow 3}$ Пусть в $\mathcal{N}_f$ нет пути из s в t , тогда определим разрез $\langle S, T \rangle$ как $S = \{v \mid \text{в } \mathcal{N}_f \text{ существует путь } s \rightsquigarrow v\}$, $T = V \setminus S$. Рассмотрим произвольную пару вершин $u \in S$ и $v \in T$. Заметим следующие два свойства:

1. Если $(u, v) \in E$, то $f(u, v) = c(u, v)$, так как иначе $c_f(u, v) = c(u, v) - f(u, v) > 0$ и в $\mathcal{N}_f$ есть ребро (u, v) , а значит v достижима из s , что неверно по построению T .
2. Если $(v, u) \in E$, то $f(v, u) = 0$, так как иначе $c_f(u, v) = f(v, u) > 0$ и в $\mathcal{N}_f$ есть ребро (u, v) . Тогда, как и выше, из $u \in S$ следовало бы $v \in S$ — противоречие.

Рассмотрим поток через разрез $\langle S, T \rangle$ и получим требуемое:

$$f(S, T) = \underbrace{\sum_{u \in S} \sum_{v \in T} f(u, v)}_{=\sum \sum c(u, v)} - \underbrace{\sum_{u \in S} \sum_{v \in T} f(v, u)}_{=\sum \sum 0=0} = \sum_{u \in S} \sum_{v \in T} c(u, v) = c(S, T)$$

□

5 Алгоритм Форда — Фалкерсона

Алгоритм 5.1. Алгоритм Форда — Фалкерсона находит максимальный поток в данной сети $\mathcal{N} = (G = (V, E), c, s, t)$. Изначально поток полагается равным нулю. На каждом шаге алгоритма происходит построение остаточной сети $\mathcal{N}_f$. Если в сети $\mathcal{N}_f$ нет пути $s \rightsquigarrow t$, то по теореме Форда — Фалкерсона поток f максимален и алгоритм завершается. Иначе DFS находит путь $s \rightsquigarrow t$ в $\mathcal{N}_f$ и пропускает вдоль него поток f' , равный минимальной пропускной способности ребра на этом пути в $\mathcal{N}_f$. Поток f' прибавляется к текущему потоку f , и происходит следующий шаг алгоритма с потоком $f \leftarrow f + f'$.

Псевдокод. Приведём стандартную реализацию этого алгоритма. Заметим, что исходный граф всегда хранится как остаточная сеть, настоящий граф никогда не строится.

```

1 struct FordFulkerson {
2     struct Edge {
3         int to, rev, cap;
4         Edge(int to, int rev, int cap) : to(to), rev(rev), cap(cap) {}
5     };
6
7     int n;
8     vector<vector<Edge>> g;
9     vector<int> used;
10
11     FordFulkerson(int n) : n(n), g(n), used(n) {}
12
13     void add_edge(int v, int to, int cap) {
14         g[v].emplace_back(to, (int)g[to].size(), cap);
15         g[to].emplace_back(v, (int)g[v].size() - 1, 0);
16     }
17
18     int dfs(int v, int t, int pushed) {
19         if (v == t || pushed == 0) return pushed;
20         used[v] = 1;
21         for (auto &e : g[v]) {
22             if (used[e.to] || e.cap == 0) continue;
23             int tr = dfs(e.to, t, min(pushed, e.cap));
24             if (!tr) continue;
25             e.cap -= tr;
26             g[e.to][e.rev].cap += tr;
27             return tr;
28         }

```

```

29     return 0;
30 }
31
32 int max_flow(int s, int t) {
33     int flow = 0;
34     while (true) {
35         used.assign(n, 0);
36         int pushed = dfs(s, t, (int)2e9);
37         if (!pushed) break;
38         flow += pushed;
39     }
40     return flow;
41 }
42 };
43
44 signed main() {
45     int n, m, s, t;
46     cin >> n >> m >> s >> t;
47     --s, --t;
48     FordFulkerson ff(n);
49     for (int i = 0; i < m; ++i) {
50         int u, v, c;
51         cin >> u >> v >> c;
52         --u, --v;
53         ff.add_edge(u, v, c);
54     }
55     cout << ff.max_flow(s, t) << '\n';
56 }

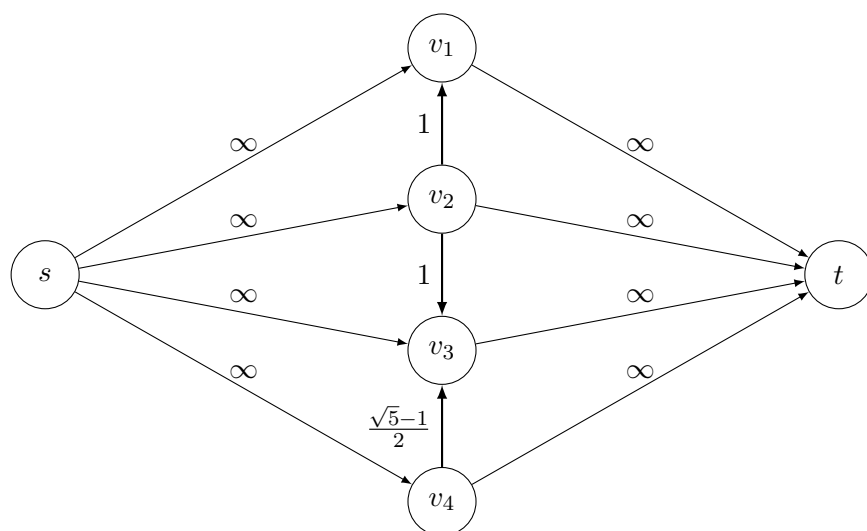
```

Теорема 5.2. В случае целочисленных пропускных способностей алгоритм Форда — Фалкерсона работает за $O(|E||f|)$, где $|f|$ — величина максимального потока.

Доказательство. Заметим, что при каждой итерации величина потока увеличивается хотя бы на 1, а каждая итерация представляет собой запуск DFS за $O(|V| + |E|)$. Таким образом, грубая оценка равна $O((|V| + |E|)|f|) = O(|E||f|)$. Почему нельзя оценить лучше? Приведём контрпример. $\square$

Теорема 5.3 (б/д). В случае вещественных пропускных способностей алгоритм Форда — Фалкерсона может не завершиться за конечное время и даже не сходиться к правильному ответу.

Доказательство. Для доказательства теоремы достаточно привести контрпример. В приведённой ниже сети пропускная способность ребра (v_4, v_3) равна $\frac{\sqrt{5}-1}{2}$. На ней можно привести пример «неудачных» выборов увеличивающих путей, когда алгоритм Форда — Фалкерсона работает бесконечно и не сходится к правильному ответу.



□